

=====

CAPSTONE PROJECT

NOTEBOOK 35: STRUCTURED PREDICTIONS FOR EXPANDED AUDIO SPLIT

=====

Purpose:

This notebook applies the frozen final
structured multi-label

model to the exact expanded validation and
test tracks used

in Notebook 34.

The goal is to:

1. Load the frozen final structured model
and scaler
2. Rebuild the structured feature matrix
consistently
3. Align features to the exact expanded
audio validation/test sets

4. Generate structured scores and pseudo-probabilities

5. Apply the frozen structured decoding rule

6. Save aligned outputs for expanded hybrid fusion

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import joblib
import numpy as np
import pandas as pd

from sklearn.metrics import (
    f1_score,
    hamming_loss,
    accuracy_score,
    precision_score,
    recall_score
)

SEED = 42
np.random.seed(SEED)

print("Seed set to:", SEED)
```

Seed set to: 42

In [2]:

```
# =====
# 2. LOAD FROZEN STRUCTURED ARTIFACTS
# =====

best_model =
joblib.load("../models/final_structured_multilabel_candidate150_best_model.joblib")
scaler =
```

```

joblib.load("../models/final_structured_multilabel_candidate150_scaler.joblib")

best_thresholds_per_label = np.load(

    "../data/processed/final_structured_multilabel_candidate150_best_thresholds_per_label.npy"
)

with
open("../data/processed/final_structured_multilabel_candidate150_best_model_name.txt", "r") as f:
    best_model_name = f.read().strip()

with
open("../data/processed/final_structured_multilabel_candidate150_best_cap.txt", "r") as f:
    best_cap_value = f.read().strip()

structured_label_cols = np.load(
    "../data/processed/structured_multilabel_candidate150_label_columns.npy",
    allow_pickle=True
)

print("Best structured model:", best_model_name)
print("Best cap value:", best_cap_value)
print("Number of structured labels:", len(structured_label_cols))
print("Threshold vector shape:", best_thresholds_per_label.shape)

```

Best structured model: OVR SGD Hinge
 Best cap value: None
 Number of structured labels: 150
 Threshold vector shape: (150,)

In [3]:

```

# =====
# 3. LOAD EXPANDED AUDIO SPLITS AND AUDIO ALIGNMENT ARRAYS
# =====

val_expanded_df =
pd.read_csv("../data/processed/audio_multilabel_candidate150_expanded_val.csv")
test_expanded_df =
pd.read_csv("../data/processed/audio_multilabel_candidate150_expanded_test.csv")

audio_label_cols = np.load(
    "../data/processed/audio_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

audio_val_track_ids =
np.load("../data/processed/audio_multilabel_candidate150_expanded_val_track_ids.npy")
audio_test_track_ids =
np.load("../data/processed/audio_multilabel_candidate150_expanded_test_track_ids.npy")

```

```

audio_y_val =
np.load("../data/processed/audio_multilabel_candidate150_expanded_y_val.npy")
audio_y_test =
np.load("../data/processed/audio_multilabel_candidate150_expanded_y_test.npy")

print("Expanded validation split shape:", val_expanded_df.shape)
print("Expanded test split shape:", test_expanded_df.shape)
print("Audio label columns:", len(audio_label_cols))
print("Audio val track IDs shape:", audio_val_track_ids.shape)
print("Audio test track IDs shape:", audio_test_track_ids.shape)
print("Audio y_val shape:", audio_y_val.shape)
print("Audio y_test shape:", audio_y_test.shape)

```

```

Expanded validation split shape: (1500, 158)
Expanded test split shape: (1500, 158)
Audio label columns: 150
Audio val track IDs shape: (1500,)
Audio test track IDs shape: (1500,)
Audio y_val shape: (1500, 150)
Audio y_test shape: (1500, 150)

```

In [4]:

```

# =====
# 4. VERIFY LABEL AND TRACK ALIGNMENT
# =====

print("Structured and audio label columns identical:",
      np.array_equal(structured_label_cols, audio_label_cols))

assert np.array_equal(structured_label_cols, audio_label_cols), \
    "Structured and audio label columns do not match."

print("Expanded validation track IDs match saved audio IDs:",
      np.array_equal(val_expanded_df["track_id"].values, audio_val_track_ids))

print("Expanded test track IDs match saved audio IDs:",
      np.array_equal(test_expanded_df["track_id"].values, audio_test_track_ids))

assert np.array_equal(val_expanded_df["track_id"].values, audio_val_track_ids), \
    "Expanded validation track IDs do not match saved audio track IDs."

assert np.array_equal(test_expanded_df["track_id"].values, audio_test_track_ids), \
    "Expanded test track IDs do not match saved audio track IDs."

```

```

Structured and audio label columns identical: True
Expanded validation track IDs match saved audio IDs: True
Expanded test track IDs match saved audio IDs: True

```

In [5]:

```

# =====
# 5. LOAD FULL CANDIDATE MASTER TABLE AND FEATURES
# =====

```

```

master_df = pd.read_csv("../data/processed/multilabel_candidate_master_table.csv")

features = pd.read_csv(
    "../data/raw/metadata/features.csv",
    header=[0, 1, 2],
    index_col=0
)
features.index = features.index.astype(int)

print("Candidate master shape:", master_df.shape)
print("Features shape:", features.shape)

display(master_df.head())

```

Candidate master shape: (81574, 157)

Features shape: (106574, 518)

	track_id	split	subset	genre_top	title	audio_path	ai
0	20	training	large	NaN	Spiritual Level	../data/raw/audio/fma_large\000\000020.mp3	
1	26	training	large	NaN	Where is your Love?	../data/raw/audio/fma_large\000\000026.mp3	
2	30	training	large	NaN	Too Happy	../data/raw/audio/fma_large\000\000030.mp3	
3	46	training	large	NaN	Yosemite	../data/raw/audio/fma_large\000\000046.mp3	
4	48	training	large	NaN	Light of Light	../data/raw/audio/fma_large\000\000048.mp3	

5 rows × 157 columns

In [6]:

```

# =====
# 6. REBUILD THE USABLE STRUCTURED FEATURE TABLE
# =====

label_cols = list(structured_label_cols)

master_df["candidate_label_count"] = master_df[label_cols].sum(axis=1)

usable_df = master_df[
    master_df["candidate_label_count"] > 0
].copy().reset_index(drop=True)

usable_df["track_id"] = usable_df["track_id"].astype(int)

track_ids = usable_df["track_id"].tolist()
features_aligned = features.reindex(track_ids).copy()

```

```

missing_feature_rows = features_aligned.isna().all(axis=1)
num_missing = int(missing_feature_rows.sum())

print("Usable rows:", usable_df.shape)
print("Requested feature rows:", len(track_ids))
print("Rows completely missing from features:", num_missing)

if num_missing > 0:
    usable_df = usable_df.loc[~missing_feature_rows].reset_index(drop=True)
    features_aligned = features_aligned.loc[~missing_feature_rows].copy()

features_aligned = features_aligned.reset_index(drop=True)

print("Aligned features shape:", features_aligned.shape)
print("Usable dataframe shape after alignment:", usable_df.shape)

```

Usable rows: (79343, 158)
 Requested feature rows: 79343
 Rows completely missing from features: 0
 Aligned features shape: (79343, 518)
 Usable dataframe shape after alignment: (79343, 158)

In [7]:

```

# =====
# 7. FLATTEN AND CLEAN FEATURES EXACTLY AS BEFORE
# =====

features_aligned.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_aligned.columns.to_flat_index()
]

X_numeric = features_aligned.copy()
X_numeric = X_numeric.select_dtypes(include=["number"])
X_numeric = X_numeric.replace([np.inf, -np.inf], np.nan)
X_numeric = X_numeric.fillna(X_numeric.mean())
X_numeric = X_numeric.astype(np.float32)

X_full = X_numeric.copy()
X_full.index = usable_df["track_id"].astype(int).values

print("Clean full structured feature matrix shape:", X_full.shape)
display(X_full.head())

```

Clean full structured feature matrix shape: (79343, 518)

	chroma_cens_kurtosis_01	chroma_cens_kurtosis_02	chroma_cens_kurtosis_03	chroma_cens_k
20	-0.193837	-0.198527	0.201546	
26	-0.699535	-0.684158	0.048825	
30	-0.721487	-0.848560	0.890904	
46	-0.119708	-0.858814	2.362546	
48	-1.054053	0.932339	0.528064	

5 rows × 518 columns

In [8]:

```
# =====
# 8. EXTRACT STRUCTURED FEATURES FOR THE EXACT EXPANDED AUDIO TRACKS
# =====

val_track_ids = val_expanded_df["track_id"].astype(int).values
test_track_ids = test_expanded_df["track_id"].astype(int).values

X_val_structured = X_full.loc[val_track_ids].copy()
X_test_structured = X_full.loc[test_track_ids].copy()

print("Structured expanded validation feature shape:", X_val_structured.shape)
print("Structured expanded test feature shape:", X_test_structured.shape)

assert np.array_equal(X_val_structured.index.values, val_track_ids), \
    "Expanded validation structured features are not aligned to track order."

assert np.array_equal(X_test_structured.index.values, test_track_ids), \
    "Expanded test structured features are not aligned to track order."
```

Structured expanded validation feature shape: (1500, 518)

Structured expanded test feature shape: (1500, 518)

In [9]:

```
# =====
# 9. SCALE FEATURES USING THE SAVED STRUCTURED SCALER
# =====

X_val_scaled = scaler.transform(X_val_structured).astype(np.float32)
X_test_scaled = scaler.transform(X_test_structured).astype(np.float32)

print("Scaled expanded validation feature shape:", X_val_scaled.shape)
print("Scaled expanded test feature shape:", X_test_scaled.shape)
```

Scaled expanded validation feature shape: (1500, 518)

Scaled expanded test feature shape: (1500, 518)

In [10]:

```

# =====
# 10. SCORE EXTRACTION AND DECODING HELPERS
# =====

def get_score_matrix(model, X_input):
    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_input)
    elif hasattr(model, "decision_function"):
        scores = model.decision_function(X_input)
    else:
        raise ValueError("Model does not support predict_proba or
decision_function.")
    return np.asarray(scores)

def scores_to_pseudoprobs(score_matrix):
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))

def decode_per_label_threshold(score_matrix, thresholds,
ensure_at_least_one=False):
    y_pred = np.zeros_like(score_matrix, dtype=np.uint8)

    for j in range(score_matrix.shape[1]):
        y_pred[:, j] = (score_matrix[:, j] >= thresholds[j]).astype(np.uint8)

    if ensure_at_least_one:
        row_sums = y_pred.sum(axis=1)
        zero_rows = np.where(row_sums == 0)[0]

        if len(zero_rows) > 0:
            top_idx = np.argmax(score_matrix[zero_rows], axis=1)
            y_pred[zero_rows, top_idx] = 1

    return y_pred

def apply_label_cap(score_matrix, y_pred, max_labels=None,
ensure_at_least_one=False):
    y_final = y_pred.copy()

    if max_labels is not None:
        for i in range(y_final.shape[0]):
            pos_idx = np.where(y_final[i] == 1)[0]

            if len(pos_idx) > max_labels:
                pos_scores = score_matrix[i, pos_idx]
                keep_order = np.argsort(-pos_scores)[:max_labels]
                keep_idx = pos_idx[keep_order]

                y_final[i, :] = 0
                y_final[i, keep_idx] = 1

    if ensure_at_least_one:
        row_sums = y_final.sum(axis=1)
        zero_rows = np.where(row_sums == 0)[0]

```



```

        if len(zero_rows) > 0:
            top_idx = np.argmax(score_matrix[zero_rows], axis=1)
            y_final[zero_rows, top_idx] = 1

    return y_final

def evaluate_multilabel(y_true, y_pred, name="Model"):
    return {
        "Model": name,
        "Micro F1": f1_score(y_true, y_pred, average="micro", zero_division=0),
        "Macro F1": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "Samples F1": f1_score(y_true, y_pred, average="samples", zero_division=0),
        "Micro Precision": precision_score(y_true, y_pred, average="micro",
zero_division=0),
        "Micro Recall": recall_score(y_true, y_pred, average="micro",
zero_division=0),
        "Hamming Loss": hamming_loss(y_true, y_pred),
        "Subset Accuracy": accuracy_score(y_true, y_pred),
        "Avg Predicted Labels": float(y_pred.sum(axis=1).mean())
    }

```

In [11]:

```

# =====
# 11. GENERATE RAW STRUCTURED SCORES AND PSEUDO-PROBABILITIES
# =====

val_scores_structured = get_score_matrix(best_model, X_val_scaled)
test_scores_structured = get_score_matrix(best_model, X_test_scaled)

val_probs_structured = scores_to_pseudoprobs(val_scores_structured)
test_probs_structured = scores_to_pseudoprobs(test_scores_structured)

print("Structured expanded validation score shape:", val_scores_structured.shape)
print("Structured expanded test score shape:", test_scores_structured.shape)
print("Structured expanded validation pseudo-prob shape:",
val_probs_structured.shape)
print("Structured expanded test pseudo-prob shape:", test_probs_structured.shape)

```

Structured expanded validation score shape: (1500, 150)

Structured expanded test score shape: (1500, 150)

Structured expanded validation pseudo-prob shape: (1500, 150)

Structured expanded test pseudo-prob shape: (1500, 150)

In [12]:

```

# =====
# 12. BUILD STRUCTURED PREDICTIONS USING THE FROZEN DECODING RULE
# =====

base_val_pred = decode_per_label_threshold(
    val_scores_structured,
    best_thresholds_per_label,

```

```

        ensure_at_least_one=True
    )

    base_test_pred = decode_per_label_threshold(
        test_scores_structured,
        best_thresholds_per_label,
        ensure_at_least_one=True
    )

    if best_cap_value == "None":
        val_pred_structured = base_val_pred.copy()
        test_pred_structured = base_test_pred.copy()
    else:
        val_pred_structured = apply_label_cap(
            val_scores_structured,
            base_val_pred,
            max_labels=int(best_cap_value),
            ensure_at_least_one=True
        )
        test_pred_structured = apply_label_cap(
            test_scores_structured,
            base_test_pred,
            max_labels=int(best_cap_value),
            ensure_at_least_one=True
        )

    print("Structured expanded validation prediction shape:",
          val_pred_structured.shape)
    print("Structured expanded test prediction shape:", test_pred_structured.shape)

```

Structured expanded validation prediction shape: (1500, 150)

Structured expanded test prediction shape: (1500, 150)

In [13]:

```

# =====
# 13. VERIFY TARGETS AGAINST THE EXPANDED AUDIO TARGETS
# =====

Y_val_structured = val_expanded_df[label_cols].values.astype(np.uint8)
Y_test_structured = test_expanded_df[label_cols].values.astype(np.uint8)

print("Structured expanded Y_val shape:", Y_val_structured.shape)
print("Structured expanded Y_test shape:", Y_test_structured.shape)

print("Validation labels identical to saved expanded audio labels:",
      np.array_equal(Y_val_structured, audio_y_val))

print("Test labels identical to saved expanded audio labels:",
      np.array_equal(Y_test_structured, audio_y_test))

assert np.array_equal(Y_val_structured, audio_y_val), \
    "Expanded validation labels do not match saved audio labels."

```

```
assert np.array_equal(Y_test_structured, audio_y_test), \
    "Expanded test labels do not match saved audio labels."
```

Structured expanded Y_val shape: (1500, 150)

Structured expanded Y_test shape: (1500, 150)

Validation labels identical to saved expanded audio labels: True

Test labels identical to saved expanded audio labels: True

In [14]:

```
# =====
# 14. EVALUATE STRUCTURED MODEL ON THE EXACT EXPANDED AUDIO SPLIT
# =====

val_results_structured = evaluate_multilabel(
    Y_val_structured,
    val_pred_structured,
    name="Structured Model on Expanded Audio Split - Validation"
)

test_results_structured = evaluate_multilabel(
    Y_test_structured,
    test_pred_structured,
    name="Structured Model on Expanded Audio Split - Test"
)

print("Structured validation results on exact expanded audio split:")
print(val_results_structured)

print("\nStructured test results on exact expanded audio split:")
print(test_results_structured)
```

Structured validation results on exact expanded audio split:

```
{'Model': 'Structured Model on Expanded Audio Split - Validation', 'Micro F1':
0.1672950414727372, 'Macro F1': 0.08146183539073083, 'Samples F1':
0.16772429772639366, 'Micro Precision': 0.09869309117683152, 'Micro Recall':
0.5486952072317925, 'Hamming Loss': 0.14233333333333334, 'Subset Accuracy': 0.0, 'Avg
Predicted Labels': 21.730666666666668}
```

Structured test results on exact expanded audio split:

```
{'Model': 'Structured Model on Expanded Audio Split - Test', 'Micro F1':
0.16369995301978388, 'Macro F1': 0.08234609476995289, 'Samples F1':
0.16479886498432983, 'Micro Precision': 0.09578204697474114, 'Micro Recall':
0.5627130809258927, 'Hamming Loss': 0.14240888888888889, 'Subset Accuracy': 0.0, 'Avg
Predicted Labels': 21.827333333333332}
```

In [15]:

```
# =====
# 15. SAVE ALIGNED OUTPUTS FOR EXPANDED HYBRID FUSION
# =====

os.makedirs("../data/processed", exist_ok=True)

pd.DataFrame([val_results_structured]).to_csv(
```

```
    "../data/processed/structured_on_expanded_audio_candidate150_val_results.csv",
    index=False
)

pd.DataFrame([test_results_structured]).to_csv(
    "../data/processed/structured_on_expanded_audio_candidate150_test_results.csv",
    index=False
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_val_scores.npy",
    val_scores_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_test_scores.npy",
    test_scores_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_val_probs.npy",
    val_probs_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_test_probs.npy",
    test_probs_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_val_pred.npy",
    val_pred_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_test_pred.npy",
    test_pred_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_y_val.npy",
    Y_val_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_y_test.npy",
    Y_test_structured
)

np.save(
    "../data/processed/structured_on_expanded_audio_candidate150_val_track_ids.npy",
    val_track_ids
)

np.save(
```

```

"../data/processed/structured_on_expanded_audio_candidate150_test_track_ids.npy",
    test_track_ids
)

np.save(

"../data/processed/structured_on_expanded_audio_candidate150_label_columns.npy",
    np.array(label_cols)
)

with
open("../data/processed/structured_on_expanded_audio_candidate150_best_model_name.txt", "w") as f:
    f.write(best_model_name)

print("Saved structured outputs aligned to the expanded audio split.")

```

Saved structured outputs aligned to the expanded audio split.

In [16]:

```

# =====
# 16. INTERPRETATION NOTES
# =====

print("1. This notebook does not retrain the structured model.")
print("2. It applies the frozen final structured baseline to the exact expanded validation and test tracks used by Notebook 34.")
print("3. Raw structured scores and sigmoid-based pseudo-probabilities were both saved for expanded hybrid fusion.")
print("4. The saved arrays are now aligned by track order, label order, and target matrix.")
print("5. The next notebook can now perform clean expanded hybrid multi-label fusion.")

```

1. This notebook does not retrain the structured model.
2. It applies the frozen final structured baseline to the exact expanded validation and test tracks used by Notebook 34.
3. Raw structured scores and sigmoid-based pseudo-probabilities were both saved for expanded hybrid fusion.
4. The saved arrays are now aligned by track order, label order, and target matrix.
5. The next notebook can now perform clean expanded hybrid multi-label fusion.